New Globe

VOL. 1, NO. 035

2026-03-30

FEATURES

Java is still available at zero-cost

Stephen Colebourne · Stephen Colebourne (Joda)

The Java ecosystem has always been built on a high quality $free (zero-cost) JDK available from Oracle, and previously Sun. This is as true today as it always has been - but the new six-monthly release cycle does mean some big changes are happening.

Six-monthly releases

Java now has a release every six months , something which greatly impacts how each version is supported. By support, I mean the provision of update releases with security patches and important bug fixes.

Up to and including Java 8, $free security updates were provided for many years. Certainly up to and beyond the launch of the next version. With Java 9 and the six-monthly release cycle, this $free support is now much more tightly controlled.

In fact, Oracle will not be providing $free long-term support (LTS) for any single Java version at all from Java 11 onwards.

Version Release date End of $free updates from Oracle Java 8 March 2014 January 2019 (for commercial use) Java 9 Sept 2017 March 2018 Java 10 March 2018 Sept 2018 Java 11 Sept 2018 March 2019 (might be extended, see below) Java 12 March 2019 Sept 2019

The idea here is simple. Oracle wants to focus its energy on moving Java forward with the cost of long-term support directly paid for by customers (instead of giving it away for $free). To do this, they need developers to continually upgrade their version of Java, moving version every six months (and picking up the patch releases in-between). Of course, for most development shops, such rapid upgrade is not feasible. But Java is now developed as OpenJDK , which means that Oracle's support dates are not the only ones to consider.

OpenJDK

A key point to grasp is that most JDK builds in the world are based on the open source OpenJDK project . The Oracle JDK is merely one of many builds that are based on the OpenJDK codebase. While it used to be the case that Oracle had additional extras in their JDK, as of Java 11 this is no longer the case.

Many other vendors also provide builds based on the OpenJDK codebase. These builds may have additional branding and/or additional non-core functionality. Most of these vendors also contribute back to the upstream OpenJDK project, including the security patches.

The impact is that the JDK you use should now be a choice you actively make , not passively accept. How fast can you get security patches? How long will it be supported? Do you need to be able to apply contractual pressure to a vendor to help with any issues?

In addition, there are two main ways that the JDK is obtained. The first is an update mechanism buit into the operating system (eg. *nix). The second is to visit a URL and download a binary (eg. Windows).

To examine this further, lets look at Java 8 and Java 11 separately.

Staying on Java 8

If you want to stay on Java 8 after January 2019, here are the choices as I see them:

1) Don't care about security.

It is entirely possible to remain on the last $free release forever. Just don't complain when hackers destroy your company.

2) Rely on Operating System updates.

On *nix platforms, you may well obtain your JDK via the operating system (eg. Red Hat , Debian , Fedora , Arch , etc.). And as such, updates to the JDK are delivered via the operating system vendor. This is where Red Hat's participation is key - they promise Java 8 updates until June 2023 in Red Hat Enterprise Linux - but they also have an "upstream first" policy, meaning they prefer to push fixes back to the "upstream" OpenJDK project. Whether you get security patches to the JDK or not will depend on your operating system vendor, and whether they need you to pay for those updates.

3) Pay for support.

A number of companies, including Azul , IBM , Oracle and Red Hat , offer ongoing support for Java. By paying them, you get access to the stream of security patches and update releases with certain guarantees (as opposed to volunteer-led approaches). If you have cash, maybe it is fair and reasonable to pay for Java?

4) Use the non-commercial build in a commercial setting.

Oracle will provide builds of Java 8 for non-commercial use until December 2020, so you could use those. But you don't want Oracle's software licensing teams chasing you, do you?

5) Build OpenJDK yourself.

The stream of security patches * is published to a public Mercurial repository under the GPL license. As such, it is perfectly possible to build OpenJDK yourself by keeping track of commits to that repository. I suspect this not a very realistic choice for most companies.

6) Use the builds from Adoptium AdoptOpenJDK .

Update 2021-11-04: AdoptOpenJDK is now Adoptium . The

pkg.go.dev is more concerned with Google's interests than good engineering

Drew DeVault

pkg.go.dev sucks. It's certainly prettier than godoc.org, but under the covers, it's a failure of engineering characteristic of the Google approach.

Go is a pretty good programming language. I have long held that this is not attributable to Google's stewardship, but rather to a small number of language designers and a clear line of influences which is drawn entirely from outside of Google — mostly from Bell Labs. pkg.go.dev provides renewed support for my argument: it has all the hallmarks of Google crapware and none of the deliberate, good engineering work that went into Go's design.

It was apparent from the start that this is what it would be. pkg.go.dev was launched as a closed-source product, justified by pointing out that godoc.org is too complex to run on an intranet, and pkg.go.dev has the same problem. There are many problems to take apart in this explanation: the assumption that the only reason an open source platform is desirable is for running it on your intranet; the unstated assumption that such complexity is necessary or agreeable in the first place; and the systemic erosion of the existing (and simple!) tools which could have been used for this purpose prior to this change. The attitude towards open source was only changed following pkg.go.dev's harsh reception by the community.

But this attitude did change, and it is open-source now 1 2 , so let's give them credit for that. The good intentions are spoilt by the fact that pkg.go.dev fetches the list of modules from proxy.golang.org : a closed-source proxy through which all of your go module fetches are being routed and tracked (oh, you didn't know? They never told you, after all). Anyway, enough of the gross disregard for the values of open source and user privacy; I do have some technical problems to talk about.

One concern comes from a blatant failure to comprehend the fundamentally decentralized nature of git hosting. Thankfully, git.sr.ht is supported now 4 — but only the git.sr.ht, i.e. the hosted instance, not the software. pkg.go.dev hard-codes a list of centralized git hosting services, and completely disregards the idea of git hosting as software rather than as a platform . Any GitLab instance other than gitlab.com (such as gitlab.freedesktop.org or salsa.debian.org); any Gogs or Gitea like Codeberg ; cgit instances like git.kernel.org ; none of these are going to work unless every host is added and the list is kept up-to-date manually. Your intranet instance of cgit? Not a chance.

They were also given an opportunity here to fix a long-standing problem with Go package discovery, namely that it requires every downstream git repository host has to (1) provide a web interface and (2) include Go-specific meta tags in the HTML. The hubris to impose your programming language 's needs onto a language-agnostic version control system! I asked: they have no interest in the better-engineered — but more worksome — approach of pursing a language agnostic design.

The worldview of the developers is whack, the new site introduces dozens of regressions, and all it really improves upon is the visual style — which could trivially have been done to godoc.org. The goal is shipping a shiny new product — not engineering a good solution. This is typical of Google's engineering ethos in general. pkg.go.dev sucks, and is added the large (and growing) body of evidence that Google is bad for Go.

Setting aside the fact that the production pkg.go.dev site is amended with closed-source patches. ↩

The GitHub comment explaining the change of heart included a link to a Google Groups discussion which requires you to log in with a Google account in order to read . 3 If you go the long way around and do some guesswork searching the archives yourself, you can find it without logging in. ↩

Commenting on Go patches also requires a Google account, by the way. ↩

But not hg.sr.ht! ↩

Free software licenses explained: MIT

Drew DeVault

This is the first in a series of posts I intend to write explaining how various free and open source software licenses work, and what that means for you as a user or developer of that software. Today we'll look at the MIT license, also sometimes referred to as the X11 or Expat license.

The MIT license is:

Both free software and open source

Permissive (and thus non-copyleft and non-viral)

This means that the license upholds the four essential freedoms of free software (the right to run, copy, distribute, study, change and improve the software) and all of the terms of the open source definition (largely the same). Further more, it is classified on the permissive/copyleft spectrum as a permissive license, meaning that it imposes relatively few obligations on the recipient of the license.

The full text of the license is quite short, so let's read it together:

The MIT License (MIT)

Copyright (c)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

The first paragraph of the license enumerates the rights which you, as a recipient of the software, are entitled to. It's this section which qualifies the license as free and open source software (assuming the later sections don't disqualify it). The key grants are the right to "use" the software (freedom 0), to "modify" and "merge" it (freedom 1), and to "distribute" and "sell" copies (freedoms 2 and 3), "without restriction". We also get some bonus grants, like the right to sublicense the software, so you could, for instance, incorporate it into a work which uses a less permissive license like the GPL.

All of this is subject to the conditions of paragraph two, of which there is only one: you must include the copyright notice and license text in any substantial copies or derivatives of the software. Thus, the MIT license requires attribution . This can be achieved by simply including the full license text (copyright notice included) somewhere in your project. For a proprietary product, this is commonly hidden away in a menu somewhere. For a free software project, where the source code is distributed alongside the product, I often include it as a comment in the relevant files. You can also add your name or the name of your organization to the list of copyright holders when contributing to MIT-licensed projects, at least in the absence of a CLA. 1

The last paragraph sets the expectations for the recipient, and it is very

important. This disclaimer of warranty is ubiquitous in nearly all free and

open source software licenses. The software is provided "as is", which is to

say, in whatever condition you found it in, for better or worse. There is no

expectation of warranty (that is to say, any support you receive is from the

goodwill of the authors and not from a contractual obligation), and there is no

guarantee of "merchantability" (that you can successfully sell it), fitness for

a particular purpose (that you can successfully use it to solve your problem),

or noninfringement (such as with respect to relevant patents). That last detail

may be of particular importance: the MIT license disclaims all liability for

patents that you might infringe upon by using the software. Other licenses often

address this case differently, such as Apache 2.0.

MIT is a good fit for projects which want to impose very few limitations on the use or reuse of the software by others. However, the permissibility of the license permits behaviors you might not like, such as creating a proprietary commercial fork of the software and selling it to others without supporting upstream. Note that the right to sell the software is an inalienable requirement of the free software

Status update, February 2020

Drew DeVault

Today I thought it'd try out something new: I have an old family recipe simmering on the stove right now, but instead of beef I'm trying out impossible beef. It cooked up a bit weird — it doesn't brown up in the same way I expect of ground beef, and it made a lot more fond than I expected. Perhaps the temperature is too high? We'll see how it fares when it's done. In the meanwhile, let's get you up to speed on my free software projects.

First, big thanks to everyone who stopped by to say "hello" at FOSDEM! Putting faces to names and getting to know everyone on a personal level is really important to me, and I would love FOSDEM even if that was all I got out of it. Got a lot of great feedback on the coming plans for SourceHut and aerc, too.

That aside, what's new? On the Wayland scene, the long-promised Sway 1.3^W1.4 was finally released, and with it wlroots 0.10.0. I've been talking it up for a while, so I won't bore you by re-listing all of the cool stuff in this release - it'll suffice to say that I think you'll enjoy it. The related tools — swayidle, swaylock, swaybg — all saw releases around the same time. The other release this month was scdoc 1.10.1, which was a simple patch release. Beyond releases, there's been some Wayland development work as well: wev received a simple bugfixes, and casa's OpenGL-based renderer rewrite has been completed nicely.

aerc progresses nicely this month as well, thanks to the support of its many dedicated contributors. Many bugfixes have landed, alongside contextual configuration options — so you can have different config settings, for example, when you have an email selected whose subject line matches a regex. A series of notmuch patches should be landing soon as well. himitsu has also seen slow progress — this pace being deliberate, as this is security-sensitive software. Several bugs have been fixed in the existing code, but there are a few more to address still. imrsh also had a little bit of improvement this month, as I started down the long road towards properly working UTF-8 support.

SourceHut improvements have also landed recently. I did some work shoring up our accessibility standards throughout the site, and SourceHut is now fully complaint with the WCAG accessibility guidelines. We now score 100% on standard performance, accessibility, and web standards compliance tests. SourceHut is the lightest weight, most usable forge. I recently fixed a bug report from a Lynx, user, too ☺ In terms of feature development, the big addition this month is support for attaching files to annotated git tags, so you can attach binaries, PGP signatures, and so on to your releases. More cool SourceHut news is coming in the post to sr.ht-announce later today.

This month's update is a little bit light on content, I'll admit. Between FOSDEM and taking some personal time, I've had less time for work this month. However, there's another reason: I have a new secret project which I've been working on. I intend to keep this project under wraps for a while still, because I don't want people to start using it before I know if it's going to pan out or not. This project is going to take a lot of time to complete, so I hope you'll bear with me for a while and trust that the results will speak for themselves. As always, thank you for your support, and I'm looking forward to another month of awesome FOSS work.

It’s not okay to pretend your software is open source

Drew DeVault

Unfortunately, I find myself writing about the Commons Clause again. For those not in the know, the Commons Clause is an addendum designed to be added to free software licenses. The restrictions it imposes (you cannot sell the software) makes the resulting franken-license nonfree. I’m not going to link to the project which brought this subject back into the discussion - they don’t deserve the referral - but the continued proliferation of software using the Commons Clause gives me reason to speak out against it some more.

One of my largest complaints with the Commons Clause is that it hijacks language used by open source projects to proliferate nonfree software, and encourages software using it to do the same. Instead of being a new software license, it tries to stick itself onto other respected licences - often the Apache 2.0 license. The name, “Commons Clause”, is also disingenuous, hijacking language used by respected entities like Creative Commons. In truth, the Commons Clause serves to remove software from the commons 1 . Combining these problems gives you language like “Apache+Commons Clause”, which is easily confused with [Apache Commons][apache-commons].

Projects using the Commons Clause have also been known to describe their license as “permissive” or “open”, some even calling their software “open source”. This is dishonest. FOSS refers to “free and open source software”. The former, free software, is defined by the free software definition , published by GNU . The latter, open source software, is defined by the open source definition , published by the OSI . Their definitions are very similar, and nearly all FOSS licenses qualify under both definitions. These are unambiguous, basic criteria which protects developers, contributors, and users of free and open source software. These definitions are so basic, important and well-respected that dismissing them is akin to dismissing climate change.

Claiming your software is open source, permissively licensed, free software, etc, when you use the Commons Clause, is lying . These lies are pervasive among users of the Commons Clause. The page listing Redis Modules , for example, states that only software under an OSI-approved license is listed. Six of the modules there are using nonfree licenses, and antirez seems content to ignore the problem until we forget about it . They’re in for a long wait - we’re not going to forget about shady, dishonest, and unethical companies like Redis Labs .

I don’t use nonfree software 2 , but I’m not going to sit here and tell you not to make nonfree software. You have every right to license your work in any way you choose. However, if you choose not to use a FOSS license, you need to own up to it. Don’t pretend that your software is something it’s not. There are many benefits to being a member of the free software community, but you are not entitled to them if your software isn’t. This behavior has to stop.

Finally, I have some praise to offer. Dgraph was briefly licensed under Apache plus the Commons Clause, and had the sort of misleading and false information this article decries on their marketing website, docs, and so on. However, they’ve rolled it back, and Dgraph is now using the Apache 2.0 license with no modifications. Thank you!

This is why I often refer to it as the “Anti-Commons Clause”, though I felt that was a bit too Stallman-esque for this article. [apache-commons]: http://commons.apache.org/ ↩

Free as in freedom, not as in free beer. ↩

Are Doctors Heroes?

Samuel Matlack · The New Atlantis

Omaha Beach Photo: Aaron Rothstein

The effervescent rays of sunshine spread their warmth across my back as I walk along Omaha Beach in Normandy. French children kick around a soccer ball, shouting and giggling across a fifty-yard stretch of sand. A tranquil ocean extends into the horizon, effortlessly mingling with the sky making it impossible to tell where one starts and the other ends. Looking out across the serene water, I imagine June 6, 1944 and the chaos that once enveloped these beaches. The young American soldiers landing here faced an onslaught of bullets from Nazi pillboxes — concrete bunkers with holes to fire through — hidden safely in the hills.

That day alone, Americans suffered two thousand and four hundred casualties . As the bodies of young men washed ashore, the fortunate survivors endured gunshot wounds while crawling up the beach amidst the blood-soaked waves. One member of the 116th Infantry Regiment said : "They're leaving us here to die like rats." D-Day provides a chilling and indelible reminder of the terror of that war and its tragic necessity; of the noble and valorous sacrifice our young heroes made to rid the world of Nazi Germany.

Normandy American Cemetery and Memorial Photo: Aaron Rothstein

I think of my trip to Omaha Beach as the 76 th anniversary of D-Day approaches and I enter the hospital each morning during the Covid-19 pandemic. "Welcome healthcare heroes!" one sign reads outside an academic hospital. "Heroes enter here!" reads another. I walk in with a mask and use a squirt of Purell on my hands, as a nurse in a gown, face shield, and mask takes my temperature. If I'm afebrile I enter the hospital. A nurse standing at the entrance shouts "Thank you, heroes!" at approaching physicians and nurses. This is not restricted to hospital entrances. Toymaker Mattel created a #ThankYouHeroes toy line of nurses, physicians, EMTs, and delivery workers. Signs hanging outside of house windows read: "Thank you essential workers for your heroism!" The quarantined populace clangs pots and pans at 7 pm throughout the city streets in honor of essential workers. From cooks to janitors to doctors: all are now heroes in the public eye . This wellspring of gratitude is well-intended and appreciated. But are those of us who work in these jobs truly heroes?

In the mid-19 th century, Thomas Carlyle, a British historian and writer, published a series of lectures in a book entitled On Heroes, Hero-worship and the Heroic in History . Though Carlyle offered some unusual theories about the role of heroes, we ought to consider the elements of his definition of a hero:

They were the leaders of men, these great ones; the modellers, patterns, and in a wide sense creators, of whatsoever the general mass of men contrived to do or to attain; all things that we see standing accomplished in the world are properly the outer material result, the practical realization and embodiment, of Thoughts that dwelt in the Great Men sent into the world: the soul of the whole world's history, it may justly be considered, were the history of these.

A hero doesn't just make a difference. A hero alters the trajectory of mankind, changes the soul of humanity, and, I think, takes a significant risk or makes a weighty sacrifice in the process. A hero is not a god but may seem god-like.

What about those who make a difference even if they're not shaping the arc of history? True, there are many who, through their professions and their actions, perform moral and selfless deeds. But, as Carlyle explains, "We see men of all kinds of professed creeds attain to almost all degrees of worth or worthlessness under each or any of them." A hero reaches even beyond such worth despite his or her imperfections.

Fortunately, there are plenty of examples throughout history of this, like the soldiers who stormed Normandy. Through their ultimate sacrifice they brought an end to Nazi genocide and sowed the seeds of freedom for millions of others. They suffered the cruel conditions at Omaha Beach and on other battlefronts to shape the world order for the better.

While physicians risk their lives during this pandemic, it is not quite the same. We come to work each day knowing that the day will end as we climb into bed, however far away from our families we are. For those of us with access to supplies, we don masks, gowns, and gloves to take care of patients with the virus and wash our hands before and after every encounter. As Dr. Greg Katz, a cardiologist in New York (and, full disclosure, my chief resident when I was an intern), writes ,

After the first few weeks of the pandemic when I had a legitimate fear for my safety due to the PPE shortage, we've largely been able to protect ourselves working in the hospital....

When we suspect a patient may have COVID, they get a designation as a PUI, or person under investigation, and are kept in an isolated room. We only enter wearing full protective equipment – N95, gowns, gloves, head covering.

When we are in a COVID unit, the equipment is even more protective, where each physician has a PAPR along with supervised donning and doffing procedures.

Don't get me wrong, it's not a risk free endeavor, but health care workers who take adequate protections have a pretty low risk of getting sick .

While we make a dramatic difference in the lives of our Covid-19 patients, we are doing what we do every day, whether there is a contagion or not: helping patients as we swore to do when we entered the profession. This is not to say that there are not heroes among us. For instance,

History will not remember us fondly

Drew DeVault

Today, we recall the Middle Ages as an unenlightened time (quite literally, in fact). We view the Middle Ages with a critical eye towards its brutality, lack of individual freedoms, and societal and technological regression. But we rarely turn that same critical lens on ourselves to consider how we'll be perceived by future generations. I expect the answer, upsetting as it may be, is this: the future will think poorly of us.

We possess the resources and production necessary to provide every human being on Earth with a comfortable living: adequate food, housing, health, and happiness. We have decided not to do so. We have achieved what one may consider the single unifying goal of the entire history of humanity: we have eliminated natural scarcity for our basic resources. We have done this, and we choose to deny our fellow humans their basic needs, in the cruel pursuit of profit. We have more empty homes than we have homeless people. America alone throws away enough food to feed the entire world population. And we choose to let our peers die of hunger and exposure.

We are politically destitute. Profits again drive everything — in the United States, Citizens United gave corporations unfettered access to buy and sell political will, and in the time since they have successfully installed politicians favorable to the elite class. Our corporations possess obscene wealth, coffers that rival those of nation-states, and rule over our people via their proxies in political office. Princeton published a study in 2014 which showed that the opinions of the average American citizen has a statistically negligible effect on political outcomes, while the opinions of the elite can all but decide the same outcomes. Our capitalist owners have unchallenged rule over society, and they rule it with the single-minded obsession to create profit at any cost, including lives.

The US Capitol was overrun by armed seditionists yesterday. Armed seditionists, who, by the way, were radicalized on the internet. As a computer engineer, I am complicit in this radicalization. The early internet was a sea of optimism, full of enthusiasm about the growing connectivity between people which had the potential to unite humanity like never before. We early adopters felt like world citizens: making friends, collaborating, and uniting with no respect for borders or ideology. What we hadn't realized is that we were also building the most powerful tool the world has ever seen for censorship, propaganda, and radicalization.

The companies which built this technology are modern slave drivers, broadly eroding worker freedoms in the first world, and in the third world seeking to exploit the cheapest slave labor they can find. We are developing technology which facilitates the authoritarian and genocidal policies of China. Anyone who speaks out is fired, corrections are quickly issued, and a statement of unconditional support for the profit generating, population murdering thugs is proclaimed. I speak passionately to my peers in my field, begging them to fight back, but many lack the courage, and most don't care — so long as their exorbitant paychecks keep coming in. Money, money, money. We are at one end of a process which launders money to wash off the blood. Morals are dead.

It's not just America — democracy is on the decline worldwide. A friend in France recently took to the streets to protest against the introduction of laws protecting the police from citizen oversight. Populist traitors tore the UK out of the EU, effective last week, dooming their people to economic and political destitution. The Greek economy has failed, right-wingers are passing discriminatory laws against LGBT Poles, and conservative populism has taken hold of much of Italy, just to name a few more. Social and political systems are regressing worldwide.

Source: Freedom House

Our entire society boils down to one measure: profit. We are being eaten alive by capitalism. Americans have been brainwashed into a national ethos which is defined by capitalism. In the relentless pursuit of profits, we have eroded all political and social freedoms and created a system defined by its remarkable cruelty in a time when we have access to greater wealth and resources than at any other time in history.

Perhaps future generations won't remember us after all, considering that in that same relentless pursuit of profits we are vigorously rendering the Earth uninhabitable. But, if they do live to remember us, they will remember us as a wicked, cruel, and unempathetic lot. We will be remembered in disgrace.

We are developing technology which facilitates the authoritarian and genocidal policies of China.

Drew DeVault

Plans for Redis 3.2

Antirez

I'm back from Paris, DotScale 2015 was a very interesting conference. Before leaving I was working on Sentinel in the context of the unstable branch: the work was mainly about connection sharing. In short, it is the ability of a few Sentinels to scale, monitoring many masters. Before to leave, and now that I'm back, I tried to "secure" a set of features that will be the basis for Redis 3.2. In the next weeks I'll be focusing developing these features, so I thought it's worth to share the list with you ASAP.

Geo hashing API: This work originated from Ardb, that was originally a fork of Redis (https://github.com/yinqiwen/ardb), and was later extracted and improved by Matt Stancliff (https://matt.sh/redis-geo) that ported it to Redis. Open source is cool eh? The code needs a refactoring effort since currently duplicates parts of the sorted set implementation. It is not impossible that I may also change a few things about the API, I'm currently not sure, if there is something to fix, I'll fix it. But the bottom line is: this is a great feature, now that Matt is no longer contributing to Redis, there is a huge risk to lose this work, so I'm going to do the effort of refactoring, reviewing and merging it as the first of the tasks for Redis 3.2. I think it is a very exciting addition to the Redis API.

Bloom filters: We'll get bloom filters in 3.2. I'm not sure if this will be implemented as a String type feature like HyperLogLog are, but more likely as a new special type, since I'm interested in non trivial semantics that are more easy to provide as a new type. I've many design ideas for bloom filters, but I'm pretty sure I would like to have the ability to control from the API the accuracy/space tradeoff, perhaps not in the lower level from of specifying number of bits and hash functions to use, but in a more higher level way. Another thing I would love to have in this API is the ability of the bloom filter to auto-depollute itself (using multiple rotating filters or something like that). I'll read all the available literature and decide what to do, but we'll get this feature into 3.2.

Memory PRs: there are two important PRs from RedisLabs to improve Redis memory usage. We'll get both merged.

Memory introspection command: A command that provides information about memory, like the LATENCY command but for memory usage. Hints about where is memory consumed, if its just the RSS that is high because of past peak memory usage, hints about amount of memory used by client output buffers, ability to resize the hash tables to save some memory if needed, and so forth.

Some Redis Cluster multi DC support. This will probably just be a "static" option of Cluster slaves so that they'll not take part to promotion when the master fails. In this way using CLUSTER FAILOVER TAKEOVER it will be possible to promote all the slaves in a minority partition.

New List type operations: A few O(1) list operations like LMERGE, and O(N) operations that will be normally used with N very small so that they are most of the times O(1) operations, like operations to move N elements from a list to another.

AOF safety feature: https://github.com/antirez/redis/pull/2574

AOF rewrites optionally using an RDB preamble, so that rewriting the AOF and reloading back the content at startup is faster.

SPOP COUNT option (already implemented, 3.2 will be the first stable versions to get it))

Redis Cluster redis-trib rebalance command, in order to automatically rehash keys to end with an more homogeneous memory usage between nodes.

A few things originally planned for 3.2 were ported to 3.0 since they were safe. A recent example is ZADD with support for options like NX and XX. In general it is possible that a few more commands about existing types will be added to Redis 3.2. This is basically a Redis version which is designed to make happy people that wanted more in the API side, since for a while we focused more on the operations aspect of Redis.

About the ETA, the work is starting Monday, and I hope they'll not take more time than the end of September, when the first RC will be pushed. Once it is RC, the RC -> Stable transition time is not scheduled, it is a function of the reporting time of critical bugs. Once for a few weeks nobody notices more bad issues, we'll go stable.

I'll follow up with new blog posts about single entries listed above, like for the Geo hashing thing, the bloom filter final implementation and API description, and so forth.

In the meantime, have fun with Redis 3.0! Comments

How to make your downstream users happy

Drew DeVault

There are a number of things that your FOSS project can be doing which will make the lives of your downstream users easier, particularly if you're writing a library or programmer-facing tooling. Many of your downstreams (Linux distros, pkgsrc, corporate users, etc) are dealing with lots of packages, and some minor tweaks to your workflow will help them out a lot.

The first thing to do is avoid using any build system or packaging system which is not the norm for your language. Also avoid incorporating information into your build which relies on being in your git repo — most packagers prefer to work with tarball snapshots, or to fetch your package from e.g. PyPI. These two issues are definitely the worst offenders. If you do have to use a custom build system, take your time to document it thoroughly, so that users who run into problems are well-equipped to address them. The typical build system or packaging process in use for your language already addressed most of those edge cases long ago, which is why we like it better. If you must fetch, say, version information from git, then please add a fallback, such as an environment variable.

Speaking of environment variables, another good one to support is SOURCE_DATE_EPOCH , for anything where the current date or time is incorporated into your build output. Many distros are striving for reproducible builds these days, which involves being able to run a build twice, or by two independent parties, and arrive at an identical checksum-verifiable result. You can probably imagine some other ways to prevent issues here — don't incorporate the full path to each file in your logs, for instance. There are more recommendations on the website linked earlier.

Though we don't like to rely on it as part of the formal packaging process, a good git discipline will also help us with the informal parts. You may already be using git tags for your releases — consider putting a changelog into your annotated tags (git tag -a). If you have good commit discipline in your project, then you can easily use git shortlog to generate such a changelog from your commit messages. This helps us understand what we can expect when upgrading, which helps incentivize us to upgrade in the first place. In How to fuck up software releases , I wrote about my semver tool, which you may find helpful in automating this process. It can also help you avoid forgetting to do things like update the version number somewhere in the code.

In short, to make your downstreams happy:

Don't rock the boat on builds and packaging.

Don't expect your code to always be in a git repo.

Consider reproducible builds.

Stick a detailed changelog in your annotated tag — which is easy if you have good commit discipline.

Overall, this is pretty easy stuff, and good practices which pay off in other respects as well. Here's a big "thanks" in advance from your future downstreams for your efforts in this regard!

The Art of Prognostication

Brendan Foht · The New Atlantis

Her oncologist sent her in to the emergency room. The diagnosis was metastatic gallbladder cancer aggressively invading her liver, resulting in liver failure. I went down to the emergency room to see her. She only spoke Bengali, so every conversation required a phone interpreter. As I walked up to the patient's bed I immediately noticed her jaundiced skin. Bilirubin, or breakdown products of red blood cells, above a certain level causes a yellowing of the eyes, the gums, and the skin where it deposits. It is frightening to see, the scarlet letter of illness.

It is unclear what the patient understood about her malady, as is so often the case when all communication occurs through an interpreter. Based on the notes in the chart, the oncologist offered chemotherapy merely as a palliative measure. No hope for cure remained, but the chemo might add weeks to the patient's life by keeping the tumor at bay. Now, however, given the patient's failing liver, any further treatment would kill the patient faster; it would hurt rather than heal.

But the patient expected chemotherapy. I told her this was unlikely but that we were going to admit her and see what we could do. Perhaps she had a treatable infection around the gallbladder, causing worsening inflammation and obstruction. Alternatively, was the cancer itself obstructing the liver's ducts? If so, the gastroenterologists or interventional radiologists could place a stent to keep the duct open.

Ultimately, though, there was nothing to fix beyond the patient's spreading cancer. No stent or antibiotics would help. Death approached, and chemotherapy would only hasten it.

Our medical team sat down with the patient and her family, using a phone interpreter, and explained the situation. But the patient, deaf to our explanations, repeatedly asked why she couldn't get chemotherapy. Each of us in the room explained the same answer in a different way — the chemotherapy would make things worse, it would kill her. In short, the oncologist was not offering chemotherapy.

The patient broke down sobbing: "give me chemotherapy!" she cried in Bengali, "I want chemotherapy. " "Please give it to me." In between the tears we were silent. She was begging for something that didn't exist — a cure. She was begging for a medication that would surely end her life.

In medical school I never learned the art of prognostication, the art of prophesying how much time was left in a patient's life. Instead we learned how to "break bad news." We role-played patient and physician and told each other about a cancer diagnosis. But we never addressed the reason any of these diagnoses were bad news. We care about the diagnosis because of the prognosis.

> *Patients and physicians alike believe that patients should have some general — albeit carefully circumscribed — awareness of death and its impending occurrence.*
>
> Brendan Foht

To be sure, it is difficult to teach the art of prognostication to medical students, who have limited knowledge and experience. How do you ask someone to prognosticate about a disease they've never seen? Moreover, different diseases follow different paths. I, for instance, am wholly unfamiliar with the typical course and treatment of lung cancer. I could not prognosticate about such a diagnosis. On the other hand, I am quite comfortable discussing the prognosis of various types of strokes. In other words, medical students have neither the knowledge nor the experience to engage in accurate prognostication. And yet, it is necessary to learn about it as early as possible, as so much of what our patients ask of us revolves around it.

In his book Death Foretold , Dr. Nicholas Christakis describes the importance of prognostication in medicine. He writes,

Predicting death is a way to counterbalance the sense of failure that arises when, despite the deployment of powerful technology in the care of the seriously ill, death cannot be prevented.... Patients and physicians alike believe that patients should have some general — albeit carefully circumscribed — awareness of death and its impending occurrence.

If a patient knows death approaches, he or she makes financial, spiritual, and filial arrangements. Such knowledge is indeed power, power to make one's last days as meaningful as possible. Moreover, a prognosis allows patients to come to terms with a diagnosis. We need time to accept our own mortality. It is not akin to getting on and off a train.

Unfortunately, as Christakis points out, "physicians regard prognosis with anxiety and disdain, and they avoid it if possible." We worry about prognosticating correctly. Many veteran physicians I admire make mistakes about a patient's course. Such uncertainty checks my own confidence; I worry about hubris, about overextending the power of my profession. Christakis writes, "The great majority of physicians, 92 percent, are 'reluctant to make predictions about a patient's illness when the clinical situation is uncertain.'" And this reluctance grounds itself in reality. In a study conducted by Dr. Christakis and Dr. Elizabeth Lamont, only 20 percent of 468 doctor's predictions were accurate. Most of these (63 percent) were overoptimistic. Fearing such inaccuracies, we avoid prognosticating altogether or hedge in our conversations with families.

According to Christakis, given its moral import, we oughtn't avoid prognosticating. But using what we know we can

A great alternative is rarely fatter than what it aims to replace

Drew DeVault

This is not always true, but in my experience, it tends to hold up. We often build or evaluate tools which aim to replace something kludgy^Wvenerable. Common examples include shells, programming languages, system utilities, and so on. Rust, Zig, etc, are taking on C in this manner; so too does zsh, fish, and oil take on bash, which in turn takes on the Bourne shell. There are many examples.

All of these tools are fine in their own respects, but they have all failed to completely supplant the software they're seeking to improve upon. 1 What these projects have in common is that they expand on the ideas of their predecessors, rather than refining them. A truly great alternative finds the nugget of truth at the center of the idea, cuts out the cruft, and solves the same problem with less.

This is one reason I like Alpine Linux, for example. It's not really aiming to replace any distro in particular so much as it competes with the Linux ecosystem as a whole. Alpine does this by being simpler than the rest: it's the only Linux system I can fit more or less entirely in my head. Compare this to the most common approach: "let's make a Debian derivative!" It kind of worked for Ubuntu, less so for everyone else. The C library Alpine ships, musl libc , is another example: it aims to replace glibc by being leaner and meaner, and I've talked about its success in this respect before .

Go is a programming language which has done relatively well in this respect. It aimed to fill a bit of a void in the high-performance internet infrastructure systems programming niche, 2 3 and it is markedly simpler than most of the other tools in its line of work. It takes the opportunity to add a few innovations — its big risk is its novel concurrency model — but Go balances this with a level of simplicity in other respects which is unchallenged among its contemporaries, 4 and a commitment to that simplicity which has endured for years. 5

There are many other examples. UTF-8 is a simple, universal approach which smooths over the idiosyncrasies of the encoding zoo which pre-dates it, and has more-or-less rendered its alternatives obsolete. JSON has almost completely replaced XML, and its grammar famously fits on a business card. 6 On the other hand, when zsh started as a superset of bash, it crippled its ability to compete on "having less warts than bash".

Rust is more vague in its inspirations, and does not start as a superset of anything. It has, however, done a poor job of scope management, and is significantly more complex than many of the languages it competes with, notably C and Go. For this reason, it struggles to root out the hold-outs in those domains, and it suffers for the difficulty in porting it to new platforms, which limits its penetration into a lot of domains that C is still thriving in. However, it succeeds in being much simpler than C++, and I expect that it will render C++ obsolete in the coming years as such. 7

In computing, we make do with a hodge podge of hacks and kludges which, at best, approximate the solutions to the problems that computing presents us. If you start with one such hack as the basis of a supposed replacement and build more on top of it, you will inherit the warts, and you may find it difficult to rid yourself of them. If, instead, you question the premise of the software, interrogate the underlying problem it's trying to solve, and apply your insights, plus a healthy dose of hindsight, you may isolate what's right from what's superfluous, and your simplified solution just might end up replacing the cruft of yore.

Some of the listed examples have not given up and would prefer that I say something to the effect of "but the jury is still out" here. ↩

That's a lot of adjectives! ↩

More concisely, I think of Go as an "internet programming language", distinct from the systems programming languages that inspired it. Its design shines especially in this context, but its value-add is less pronounced for other tasks in the systems programming domain - compilers, operating systems, etc. ↩

The Go spec is quite concise and has changed very little since Go's inception. Go is also unique among its contemporaries for (1) writing a spec which (2) supports the development of multiple competing implementations. ↩

Past tense, unfortunately, now that Go 2 is getting stirred up. ↩

It is possible that JSON has achieved too much success in this respect, as it has found its way into a lot of use-cases for which it is less than ideal. ↩

Despite my infamous distaste for Rust, long-time readers will know that where I have distaste for Rust, I have passionate scorn for C++. I'm quite glad to see Rust taking it on, and I hope very much that it succeeds in this respect. ↩

Writing a Wayland Compositor, Part 1: Hello wlroots

Drew DeVault

This is the first in a series of many articles I'm writing on the subject of building a functional Wayland compositor from scratch. As you may know, I am the lead maintainer of sway , a reasonably popular Wayland compositor. Along with many other talented developers, we've been working on wlroots over the past few months. This is a powerful tool for creating new Wayland compositors, but it is very dense and difficult to understand. Do not despair! The intention of these articles is to make you understand and feel comfortable using it.

Before we dive in, a quick note: the wlroots team is starting a crowdfunding campaign today to fund travel for each of our core contributors to meet in person and work for two weeks on a hackathon. Please consider contributing to the campaign !

You must read and comprehend my earlier article, An introduction to Wayland , before attempting to understand this series of blog posts, as I will be relying on concepts and terminology introduced there to speed things up. Some background in OpenGL is helpful, but not required. A good understanding of C is mandatory. If you have any questions about any of the articles in this series, please reach out to me directly via sir@cmpwn.com or to the wlroots team at #sway-devel on irc.freenode.net .

During this series of articles, the compositor we're building will live on GitHub: Wayland McWayface . Each article in this series will be presented as a breakdown of a single commit between zero and a fully functional Wayland compositor. The commit for this article is f89092e . I'm only going to explain the important parts - I suggest you review the entire commit separately.

Let's get started. First, I'm going to define a struct for holding our compositor's state:

+struct mcw_server {
1. struct wl_display *wl_display;
2. struct wl_event_loop *wl_event_loop;

+};

Note: mcw is short for McWayface. We'll be using this acronym throughout the article series. We'll set one of these aside and initialize the Wayland display for it 1 :

int main(int argc, char **argv) {
1. struct mcw_server server;
2.
3. server.wl_display = wl_display_create();
4. assert(server.wl_display);
5. server.wl_event_loop
   = wl_display_get_event_loop(server.wl_display);
6. assert(server.wl_event_loop);

return 0; }

The Wayland display gives us a number of things, but for now all we care about is the event loop. This event loop is deeply integrated into wlroots, and is used for things like dispatching signals across the application, being notified when data is available on various file descriptors, and so on.

Next, we need to create the backend:

struct mcw_server { struct wl_display *wl_display; struct wl_event_loop *wl_event_loop;

1. struct wlr_backend *backend;

};

The backend is our first wlroots concept. The backend is responsible for abstracting the low level input and output implementations from you. Each backend can generate zero or more input devices (such as mice, keyboards, etc) and zero or more output devices (such as monitors on your desk). Backends have nothing to do with Wayland - their purpose is to help you with the other APIs you need to use as a Wayland compositor. There are various backends with various purposes:

The drm backend utilizes the Linux DRM subsystem to render directly to your physical displays.

The libinput backend utilizes libinput to enumerate and control physical input devices.

The wayland backend creates "outputs" as windows on another running Wayland compositors, allowing you to nest compositors. Useful for debugging.

The x11 backend is similar to the Wayland backend, but opens an x11 window on an x11 server rather than a Wayland window on a Wayland server.

Another important backend is the multi backend, which allows you to initialize several backends at once and aggregate their input and output devices. This is necessary, for example, to utilize both drm and libinput simultaneously.

wlroots provides a helper function for automatically choosing the most appropriate backend based on the user's environment:

server.wl_event_loop
= wl_display_get_event_loop(server.wl_display); assert(server.wl_event_loop);

1. server.backend
   = wlr_backend_autocreate(server.wl_display);
2. assert(server.backend);

return 0; }

I would generally suggest using either the Wayland or X11 backends during development, especially before we have a way of exiting the compositor. If you call wlr_backend_autocreate from a running Wayland or X11 session, the respective backends will be automatically cho-

User-defined literals in Java?

Stephen Colebourne · Stephen Colebourne (Joda)

Java has a number of literals for creating values, but wouldn't it be nice if we had more?

Current literals

These are some of the literals we can write in Java today:

integer - 123 , 12s , 1234L , 0xB8E817 , 077 , 0b1011_1010

floating point - 45.6f , 56.7d , 7.656e6

string - "Hello world"

char - 'a'

boolean - true , false

null - null

Project Amber is also considering adding multi-line and/or raw string literals.

But there are many other data types that would benefit from literals, such as dates, regex and URIs.

User-defined literals

In my ideal future, I'd like to see Java extended to support some form of user-defined literals. This would allow the author of a class to provide a mechanism to convert a sequence of characters into an instance of that class. It may be clearer to see some examples using one possible syntax (using backticks):

Currency currency = `GBP`; LocalDate date = `2019-03-29`; Pattern pattern = `strata\.\w+`; URI uri = `https://blog.joda.org/`;

A number of semantic features would be required:

Type inference

Raw processing

Validated at compile-time

Type inference

Type inference is of course a key aspect of literals. It would have to work in a similar way to the existing literals, but with a tweak to handle the new var keyword. ie. these two would be equivalent:

LocalDate date = `2019-03-29`; var date = LocalDate`2019-03-29`;

The type inference would also work with methods (compile error if ambiguous):

boolean inferior = isShortMonth(`2019-04-12`);

public boolean isShortMonth(LocalDate date) { return date.lengthOfMonth()

Raw processing

Processing of the literal should not be limited by Java's escape mechanisms. User-defined literals need access to the raw string. Note that this is especially useful for regex, but would also be useful for files on Windows:

/\/ user-defined literals var pattern = Pattern`strata\.\w+`; /\/ today var pattern = Pattern.compile("strata\\.\\w+");

Today, the `\` needs to be escaped, making the regex difficult to read.

Clearly, the problem with parsing raw literals is that there is no mechanism to escape. But the use cases for user-defined literals tend to have constrained formats, eg. a date doesn't contain random characters. So, although there might be edge cases where this would be a problem, they would vert much be edge cases.

Validated at Compile-time

A key feature of literals is that they are validated at compile-time. You can't use an integer literal to create an int if the value is larger than the maximum allowed integer (2^31).

User-defined literals also need to be parsed and validated at compile-time too. Thus this code would not compile:

LocalDate date = `2019-02-31`;

Most types which would benefit from literals only accept specific input formats, so being able to check this at compile time would be beneficial.

How would it be implemented?

I'm pretty confident that there are various ways it could be done. I'm not going to pick an approach, as ultimately those that control the JVM and language are better placed to decide. Clearly though, there is going to need to be some form of factory method on the user class that performs the parse, with that method invoked by the compiler. And ideally, the results of the parse would be stored in the constant pool rather than re-parsed at runtime.

What I would say is that user-defined literals would almost be a requirement for making value types usable, so something like this may be on the way anyway.

I like literals. And I would really like to be able to define my own!

Any thoughts?

Status update, September 2022

Drew DeVault

I have COVID-19 and I am halfway through my stockpile of tissues, so I'm gonna keep this status update brief.

In Hare news, I finally put the last pieces into place to make cross compiling as easy as possible. Nothing else particularly world-shattering going on here. I have a bunch of new stuff in my patch queue to review once I'm feeling better, however, including bigint stuff — a big step towards TLS support. Unrelatedly, TLS support seems to be progressing upstream in qbe. (See what I did there?)

powerctl is a small new project I wrote to configure power management states on Linux. I'm pretty pleased with how it turned out. It makes for a good case study on Hare for systems programming.

In Helios, I have been refactoring the hell out of everything, rewriting or redesigning large parts of it from scratch. Presently this means that a lot of the functionality which was previously present was removed, and is being slowly re-introduced with substantial changes. The key is reworking these features to take better consideration of the full object lifecycle — creating, copying, and destroying capabilities. An improvement which ended up being useful in the course of this work is adding address space IDs (PCIDs on x86_64), which is going to offer a substantial performance boost down the line.

Alright, time for a nap. Bye!

Redis will remain BSD licensed

Antirez

Today a page about the new Common Clause license in the Redis Labs web site was interpreted as if Redis itself switched license. This is not the case, Redis is, and will remain, BSD licensed. However in the era of [edit] uncontrollable spreading of information, my attempts to provide the correct information failed, and I'm still seeing everywhere "Redis is no longer open source". The reality is that Redis remains BSD, and actually Redis Labs did the right thing supporting my effort to keep the Redis core open as usually.

What is happening instead is that certain Redis modules, developed inside Redis Labs, are now released under the Common Clause (using Apache license as a base license). This means that basically certain enterprise add-ons, instead of being completely closed source as they could be, will be available with a more permissive license.

I think that Redis Labs Common Clause page did not provide a clear and complete information, but software companies often make communication errors, it happens. To me however, it looks more important that while running a system software business in the "cloud era" (LOL) is very challenging using an open source license, yet Redis Labs totally understood and supported the idea that the Redis core is an open source project, in the *most permissive license ever*, that is, BSD, and during the years provided a lot of funding to the project.

The reason why certain modules developed internally at Redis Labs are switching license, is because they are added value that Redis Labs wants to be able to provide only to end users that are willing to compile and install the system themselves, or to the Redis Labs customers using their services.

IN BRIEF

Shifter, Serverless Blog: Learn how Shifter transforms WordPress blogs and websites into static sites to make them faster, more secure and scalable in this guest post.

Verne Lindner, Serverless Blog: Tag your Lambdas to track errors and debug serverless applications. If you're using NodeJS or Python, we'll help you find even the trickiest serverless application errors faster.

Stefanie Monge, Serverless Blog: How Serverless Partner AbstractAI leveraged the Serverless Framework and Lambda to reduce the cost of running back-end services by 95%.

Rustem Feyzkhanov, Serverless Blog: We'll cover how to use TensorFlow, the Serverless Framework, AWS Lambda and API Gateway to deploy a simple deep learning model.

Alex DeBrie, Serverless Blog: Learn how to set up a custom domain name for AWS Lambda & API Gateway using the Serverless Framework to configure a clean domain name for your services.

Asanka Nissanka, Serverless Blog: Why we decided to migrate our services running on docker containers to serverless stack using aws lambda functions and aws api gateway

Austen Collins, Serverless Blog: Forget infrastructure. We're giving you a new option to deploy serverless use-cases easily — without managing complex infrastructure configuration files.

Unrelatedly, TLS support seems to be progressing upstream in qbe.

Drew DeVault

My plans at FOSDEM: SourceHut, Hare, and Helios

Drew DeVault

FOSDEM is right around the corner, and finally in person after long years of dealing with COVID. I'll be there again this year, and I'm looking forward to it! I have four slots on the schedule (wow! Thanks for arranging these, FOSDEM team) and I'll be talking about several projects. There is a quick lightning talk on Saturday to introduce Helios and tease a full-length talk on Sunday, a meetup for the Hare community, and a meetup for the SourceHut community. I hope to see you there!

Lightning talk: Introducing Helios

Saturday 12:00 at H.2215 (Ferrer)

Helios is a simple microkernel written in part to demonstrate the applicability of the Hare programming language to kernels. This talk briefly explains why Helios is interesting and is a teaser for a more in-depth talk in the microkernel room tomorrow.

Hare is a systems programming language designed to be simple, stable, and robust. Hare uses a static type system, manual memory management, and a minimal runtime. It is well-suited to writing operating systems, system tools, compilers, networking software, and other low-level, high performance tasks. Helios uses Hare to implement a microkernel, largely inspired by seL4.

BoF: The Hare programming language

Saturday 15:00 at UB2.147

Hare is a systems programming language designed to be simple, stable, and robust. Hare uses a static type system, manual memory management, and a minimal runtime. It is well-suited to writing operating systems, system tools, compilers, networking software, and other low-level, high performance tasks.

At this meeting we'll sum up the state of affairs with Hare, our plans for the future, and encourage discussions with the community. We'll also demonstrate a few interesting Hare projects, including Helios, a micro-kernel written in Hare, and encourage each other to work on interesting projects in the Hare community.

BoF: SourceHut meetup

Saturday 16:00 at UB2.147

SourceHut is a free software forge for developing software projects, providing git and mercurial hosting, continuous integration, mailing lists, and more. We'll be meeting here again in 2023 to discuss the platform and its community, the completion of the GraphQL rollout and the migration to the EU, and any other topics on the minds of the attendees.

Sunday 13:00 at H.1308 (Rolin)

Helios is a simple microkernel written in part to demonstrate the applicability of the Hare programming language to kernels. This talk will introduce the design and rationale for Helios, address some details of its implementation, compare it with seL4, and elaborate on the broader plans for the system.

Hare uses a static type system, manual memory management, and a minimal runtime.

Drew DeVault

New Globe

Vol. 1, No. 035 · 2026-03-30

Curated and composed automatically.